

**KSCSC**

Cyber Security Consulting · Offensive Security & Cloud

Penetration Test Report — OrderFlow Platform

Sample / demonstration report · fictional multi-tenant SaaS engagement

Prepared for	Northwind Retail Group (fictional)
Prepared by	KSCSC — Krzysztof Świdrak
Engagement	Grey-box penetration test — OrderFlow platform
Report date	2026 (sample)
Version	1.0
Classification	Confidential — demonstration / fictional data

DEMONSTRATION REPORT — all data is fictional. Client (Northwind Retail Group), application (OrderFlow), hostnames, tenants, findings, requests and screenshots are invented for illustration. It shows report structure, threat modelling, profiling, findings with example PoCs, ATT&CK mapping, detection guidance and remediation — not any real system. Replace the cover branding before use.

Contents

1. Executive Summary	4
1.1. Scope of Work	4
1.2. Objectives	5
1.3. Threat Model & Test Accounts	5
1.4. Timeline	6
2. Risk Profile & Attack Chains	7
2.1. Risk profile by theme	7
2.2. Primary attack chain — unauthenticated to full multi-tenant compromise	7
2.3. Secondary attack chain — malicious tenant to durable cross-tenant foothold	8
2.4. Residual risk, coverage & limitations	9
3. Reconnaissance & Profiling	9
3.1. Passive attack-surface enumeration	9
3.2. Architecture map (derived, black-box)	9
3.3. Transport & header posture	10
4. Methodology	11
5. Detailed Findings	12
5.1. F-001 — Cross-tenant BOLA: any tenant can read/modify another tenant's orders	13
5.2. F-002 — Broken function-level authorisation: low-privilege user reaches admin export	15
5.3. F-003 — Unauthenticated actuator endpoint discloses application secrets	17
5.4. F-004 — SSRF via webhook URL-preview (metadata backstopped)	19
5.5. F-005 — Stored XSS in order note rendered in the operator console	20
5.6. F-006 — Long-lived refresh token stored in browser localStorage	21
5.7. F-007 — Missing HTTP security headers across all origins	22
5.8. F-008 — Verbose error handling discloses stack traces	23
5.9. F-009 — Dependency & transport posture (current and patched)	24
5.10. F-010 — External attack-surface enumeration: unscoped assets	25
6. MITRE ATT&CK; Mapping	26
7. Indicators of Compromise & Detection Opportunities	27
8. Remediation Roadmap	28

9. Appendix A — Tools Used	29
10. Appendix B — Glossary	29

Penetration Test Report — OrderFlow Platform

Demonstration report — fictional data. This document is a sample deliverable built around an invented client (**Northwind Retail Group**) and an invented application (**OrderFlow**). All hostnames, tenants, findings, requests, and screenshots are fabricated for illustration. It exists to show report structure, depth, and the value a client receives — not to describe any real system. Replace the cover branding with your own before use.

1. Executive Summary

This report presents the results of a grey-box penetration test of **OrderFlow**, a multi-tenant B2B ordering and fulfilment platform operated by Northwind Retail Group, together with its public web application, authenticated operator console, and REST API. The objective was to assess the security posture of the in-scope systems, identify vulnerabilities exploitable by unauthenticated or low-privilege authenticated users, and — as the engagement's designated high-value class — determine whether **tenant isolation** holds consistently across the platform.

The assessment identified **one Critical, three High, three Medium, one Low, and two Informational** findings. The dominant theme is inconsistent server-side authorisation: the platform enforces authentication well but does not consistently verify, per request, that the calling tenant *owns* the object or is *entitled* to the function being invoked. The single most serious issue (F-001) lets any authenticated tenant read and modify another tenant's orders — a direct breach of the multi-tenant boundary that is the product's core security promise. A parallel weakness (F-003) exposes application secrets to the public internet with no authentication at all.

Risk distribution

Critical	High	Medium	Low	Informational
1	3	3	1	2

Risk heat map (Likelihood × Impact)

Impact \ Likelihood	Low	Medium	High
High	—	F-004	F-001 , F-002, F-003
Medium	—	F-005, F-006	F-008
Low	—	F-009	F-007, F-010

Bottom line for the business. OrderFlow's infrastructure, transport security, and dependency hygiene are in good shape. The exposure is concentrated in the application authorisation layer and in one operational-endpoint misconfiguration. None of the top findings requires advanced skill to exploit — F-001 and F-003 are reachable with a single crafted request. We recommend treating F-001 and F-003 as incident-grade: remediate immediately, then review logs for prior exploitation using the indicators in Section 7. The remaining items are a focused, achievable hardening programme (Section 8).

Crucially, the per-finding severities above **understate** the exposure, because the findings chain: an unauthenticated attacker can combine two "High" findings into a Critical, full-dataset breach.

Section 2 presents the risk profile and the attack chains — it is the fastest way to understand what this assessment actually means for the business.

1.1. Scope of Work

Asset	Description	Approach
app.orderflow.example	Customer web application (Next.js SPA)	Grey-box, authenticated
console.orderflow.example	Operator / admin console	Grey-box, authenticated
api.orderflow.example	REST API v2 (Node/Spring services)	Grey-box, authenticated
auth.orderflow.example	SSO / identity provider (Keycloak)	Passive / OIDC review
AWS data plane	S3, API Gateway, CloudFront, ALB	Access-control review only

Out of scope: denial-of-service testing, social engineering, physical security, and any host not listed above. Testing was conducted against the staging environment, which mirrors production configuration.

1.2. Objectives

1. Validate that **tenant isolation** (object- and function-level authorisation) holds across the API and both front ends.
2. Assess the two contracted threat models: an unauthenticated external attacker, and an authenticated low-privilege tenant user.
3. Review session/token design, input-validation posture, transport and header hardening, and the access-control posture of the supporting AWS surface.
4. Provide prioritised, actionable remediation and detection guidance the client can operationalise.

1.3. Threat Model & Test Accounts

The engagement was executed against three explicit scenarios, escalating in assumed access:

1. **Unauthenticated external attacker** — no credentials; probing the public attack surface (drove F-003, F-010).
2. **Low-privilege authenticated user, vertical escalation** — a viewer/operator attempting to reach higher-privilege functions within their own tenant (drove F-002).
3. **Authenticated user, horizontal / cross-tenant escalation** — a user of tenant A attempting to read or act on tenant B's data (drove F-001, the engagement's high-value class).

To exercise the cross-tenant scenario, accounts were provisioned in two separate tenants. In each authorisation test a **victim** account owns the target object and a **challenger** account — deliberately in a different tenant — attempts to reach it.

Test login	Tenant	Role	Role in testing
alice@acme-foods	acme-foods	operator	Victim — owns target orders
carol@bravo-logistics	bravo-logistics	viewer	Challenger — cross-tenant attacker
dave@acme-foods	acme-foods	viewer	Low-privilege escalation actor
erin@bravo-logistics	bravo-logistics	tenant-admin	Entitlement/role reference

Because each login authenticates into exactly one tenant, any request in which one account reads or mutates an object owned by another account's tenant is, by definition, cross-tenant access. Where the server acts on that object without first verifying the caller owns the referenced identifier, the missing per-object ownership check is a **Broken Object Level Authorisation (BOLA)** flaw.

1.4. Timeline

Phase	Start	End
Kick-off & scoping	Day 0	Day 0
Passive reconnaissance & profiling	Day 1	Day 2
Access verification	Day 2	Day 2
Active testing / exploitation	Day 3	Day 8
Reporting	Day 9	Day 10
Retest (findings fixed)	On request	—

2. Risk Profile & Attack Chains

Read one at a time, the findings understate the real exposure — an attacker does not stop at a single weakness. This section rolls the findings into business risk themes, then shows the concrete chains by which they combine into full compromise. It is the part of the report that answers "so how bad is this, really?"

2.1. Risk profile by theme

The ten findings collapse into five risk themes. Two of them carry essentially all of the platform's real risk; the rest are hygiene that matters only once the top two are fixed.

Risk theme	Rating	Driven by	What it means for the business
Tenant isolation	HIGH	F-001, F-002	The multi-tenant boundary — the product's core promise — is not enforced per request. One tenant can read, alter, or bulk-export another tenant's data.
Secrets & key management	HIGH	F-003	Application secrets, including the token-signing key, are reachable unauthenticated. Whoever holds the key can impersonate anyone.
Session & token security	MEDIUM	F-005, F-006	A stored-XSS + long-lived token-in-localStorage combination turns a transient client compromise into a durable account takeover.
Server-side request control	MEDIUM	F-004	The platform can be used as a request proxy from its trusted network position; a single egress control stands between this and cloud-credential theft.
Baseline hardening	LOW	F-007, F-008, F-010	Missing headers, verbose errors, and unscoped assets. No standalone risk, but they remove the safety nets that would contain the items above.

2.2. Primary attack chain — unauthenticated to full multi-tenant compromise

The single most important scenario needs **no credentials and no advanced skill**. It chains two individually "High" findings into a Critical outcome that neither conveys alone.

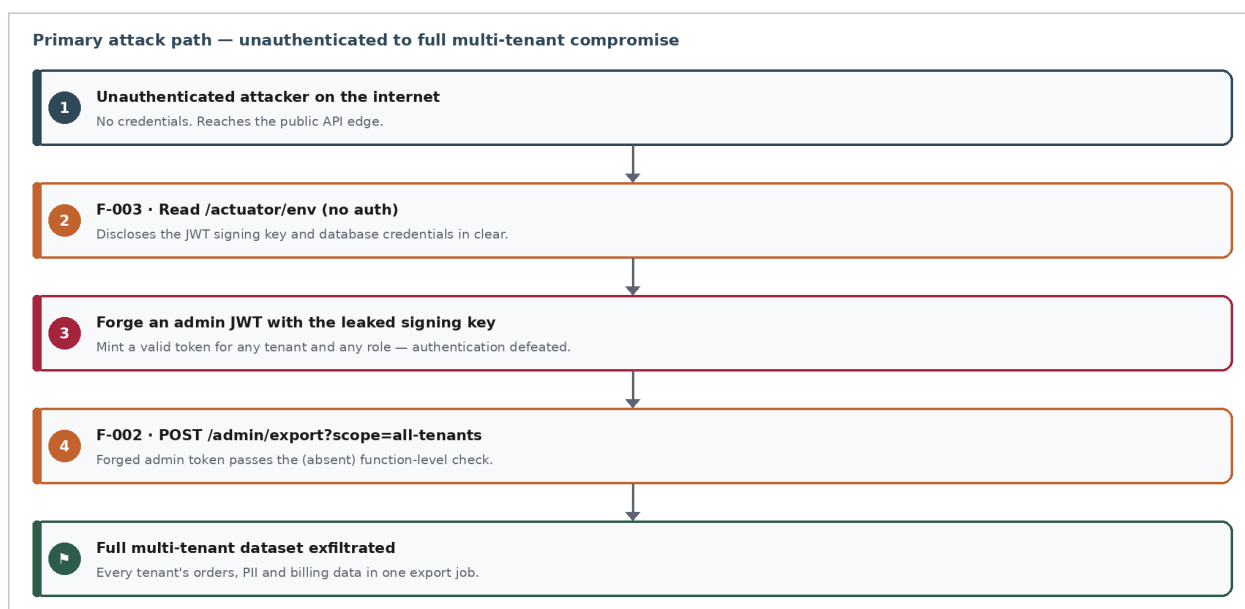


Figure — Primary attack chain — one unauthenticated request leaks the token-signing key, which defeats authentication entirely and unlocks the all-tenant export.

Step by step: an unauthenticated attacker reads `/actuator/env` (**F-003**) and obtains the JWT signing key in clear. With that key they forge a valid token for any tenant and any role — authentication is now defeated, not bypassed. They present the forged admin token to `/admin/export?scope=all-tenants` (**F-002**), which performs no function-level check, and receive a CSV of every tenant's orders, PII, and billing data.

Why the chain matters more than the parts. F-003 read alone sounds routine ("an exposed actuator endpoint"); F-002 alone requires an admin token the attacker "shouldn't" have. Chained, F-003 *supplies* that token, so the combined path is a Critical, unauthenticated, full-dataset breach. This is precisely why a per-finding severity view understates risk — and why F-003 is flagged incident-grade despite its modest-sounding description.

2.3. Secondary attack chain — malicious tenant to durable cross-tenant foothold

The second path assumes only a single low-privilege account in any tenant — the realistic "malicious customer / insider" threat actor.

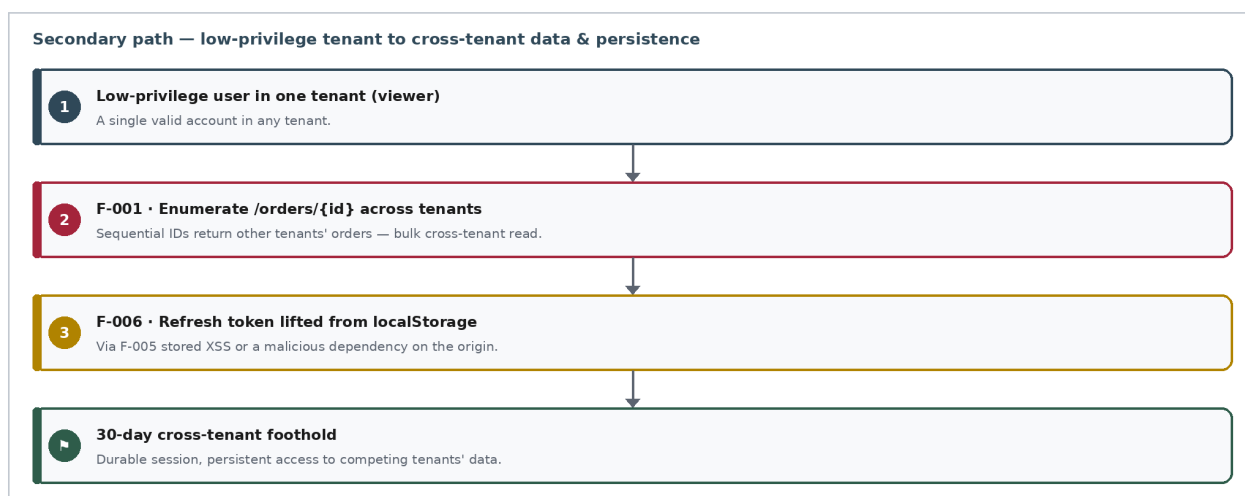


Figure — Secondary attack chain — a low-privilege tenant user reads across tenants via IDOR and, through stored XSS, lifts a 30-day refresh token for durable access.

The user enumerates `/orders/{id}` (**F-001**) and harvests other tenants' orders directly — no escalation needed. To persist, they plant a stored-XSS payload in an order note (**F-005**) that, when

an operator views it, lifts the operator's 30-day refresh token from `localStorage` (**F-006**). The result is a durable cross-tenant foothold rather than a one-off read.

2.4. Residual risk, coverage & limitations

Penetration testing is time- and scope-bounded; the absence of a finding is not proof that a component is secure. Coverage limits for this engagement:

- The unscoped `reports.*` (BI tool) and `staging.*` hosts on the same estate were **not tested** (F-010) — the reporting surface is the likeliest remaining location for aggregated cross-tenant data.
- DoS/DDoS testing and source-code review were out of scope; the mobile API was not in scope.
- Testing ran against staging; production configuration drift is possible.

Residual-risk statement. Today the platform's risk is **Critical**, dominated by the primary chain. Remediating just F-001 and F-003 (the two incident-grade items) breaks both chains and drops residual risk to **Low-Medium**, at which point the untested reporting surface becomes the leading remaining unknown. We recommend a focused retest of those two fixes plus a scoped assessment of `reports.*` (Section 8).

3. Reconnaissance & Profiling

Before any active testing, we built a picture of the external estate from public sources only — no asset was contacted intrusively in this phase. This establishes the real attack surface (which is often wider than the agreed scope) and lets us target active testing efficiently.

3.1. Passive attack-surface enumeration

Sources: public DNS, Certificate-Transparency logs, search-engine and archive data, and cloud global-namespace existence checks. This surfaced fourteen subdomains, a public S3 bucket, and — notably — two hosts not in the agreed scope: a `staging.*` origin and a `reports.*` host running an exposed BI tool.

```

Terminal — passive attack-surface enumeration

$ subfinder -d orderflow.example -silent | httpx -silent -title -tech-detect

app.orderflow.example      [200] [OrderFlow]      [Next.js, Cloudflare]
api.orderflow.example      [200] [OrderFlow API]   [nginx, AWS-ALB]
auth.orderflow.example     [200] [Sign in]         [Keycloak]
staging.orderflow.example  [401]              [nginx]           <-- unscoped
reports.orderflow.example  [200] [Metabase]        [Jetty]           <-- unscoped
assets-of-prod.s3.amazonaws.com [403] (bucket exists)

$ # CT-log mining (crt.sh) surfaced 2 hosts not in the agreed scope
[+] 14 subdomains, 1 public S3 bucket, 1 exposed BI tool (Metabase)

```

Figure — Passive subdomain and technology enumeration. Two live hosts outside the agreed scope were discovered from public data alone.

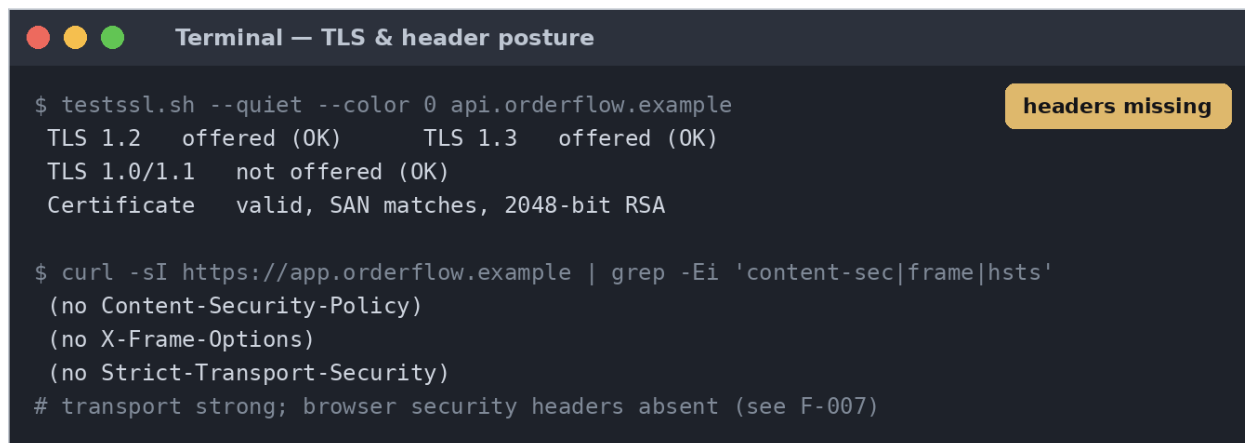
3.2. Architecture map (derived, black-box)

From response headers, OIDC discovery, JS bundle inspection, and DNS/CT correlation, the platform profiles as:

- **Edge:** CloudFront CDN in front of an application load balancer; TLS 1.2/1.3, HSTS absent on app origins.

- **Front ends:** a Next.js customer SPA (app.*) and an operator console (console.*), both talking to a shared REST API.
- **API:** REST v2 over api.*; a mix of Node and JVM (Spring Boot) services behind the ALB; a Spring Actuator surface is reachable (see F-003).
- **Identity:** Keycloak OIDC at auth.*; access + refresh tokens issued to the browser.
- **Data plane:** PostgreSQL (not directly exposed); S3 for document/asset storage; per-tenant logical separation enforced in application code rather than at the data layer — which is precisely why the authorisation findings matter.

3.3. Transport & header posture



The image shows a terminal window titled "Terminal — TLS & header posture". It contains two command-line tests. The first test, `$ testssl.sh --quiet --color 0 api.orderflow.example`, shows results for TLS 1.2 and 1.3 being offered (OK), TLS 1.0/1.1 not offered (OK), and a valid certificate with a 2048-bit RSA key. A yellow box labeled "headers missing" is positioned to the right of this test's output. The second test, `$ curl -sI https://app.orderflow.example | grep -Ei 'content-sec|frame|hsts'`, shows that Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security headers are all absent. A comment at the bottom of the terminal output states: "# transport strong; browser security headers absent (see F-007)".

```
$ testssl.sh --quiet --color 0 api.orderflow.example
TLS 1.2   offered (OK)      TLS 1.3   offered (OK)
TLS 1.0/1.1 not offered (OK)
Certificate valid, SAN matches, 2048-bit RSA

$ curl -sI https://app.orderflow.example | grep -Ei 'content-sec|frame|hsts'
(no Content-Security-Policy)
(no X-Frame-Options)
(no Strict-Transport-Security)
# transport strong; browser security headers absent (see F-007)
```

Figure — Transport security is strong (TLS 1.2/1.3, no legacy protocols); browser-side security headers are absent on the application origins — expanded in F-007.

Profiling conclusion. The platform is a modern, well-built stack whose security model depends on the *application* correctly scoping every request to the caller's tenant and role. That single assumption is where the significant findings cluster.

4. Methodology

Testing followed a structured process — **Planning** (information gathering, live-host detection, reconnaissance, fingerprinting), **Exploitation** (vulnerability assessment, enumeration, controlled exploitation), and **Reporting** (analysis, risk rating, remediation and detection guidance).

Risk is rated with the **OWASP Risk Rating Methodology**: overall severity = Likelihood × Impact, each derived from underlying factors (threat-agent skill/motive/opportunity/size; vulnerability ease-of-discovery/ease-of-exploit/awareness/detectability; technical impact to confidentiality/integrity/availability/accountability; business impact of financial/reputation/non-compliance/privacy). Each factor is scored 0-9; group averages map to Low (0-<3), Medium (3-<6), High (6-9).

5. Detailed Findings

ID	Finding	Risk	Affected system
F-001	Cross-tenant BOLA — any tenant can read/modify another tenant's orders	CRITICAL	api.* /orders
F-002	Broken function-level authorisation — low-privilege user reaches admin export	HIGH	api.* /admin/export
F-003	Unauthenticated actuator endpoint discloses application secrets	HIGH	api.* /actuator
F-004	SSRF via webhook URL-preview (metadata backstopped)	MEDIUM	api.* /webhooks/preview
F-005	Stored XSS in order note rendered in the operator console	MEDIUM	console.*
F-006	Long-lived refresh token stored in browser localStorage	MEDIUM	app. / console.
F-007	Missing HTTP security headers across all origins	LOW	all web origins
F-008	Verbose error handling discloses stack traces	LOW	api.*
F-009	Dependency & transport posture (current and patched)	INFORMATIONAL	estate
F-010	External attack-surface enumeration — unscoped assets	INFORMATIONAL	estate

5.1. F-001 — Cross-tenant BOLA: any tenant can read/modify another tenant's orders

Field	Value
Risk Rating	Critical
Calculation	High (Likelihood 7.1) × High (Impact 7.4) = Critical
Affected system	api.orderflow.example — /api/v2/orders/{orderId}
CWE	CWE-639, CWE-863

OWASP factor scores (how the Likelihood 7.1 / Impact 7.4 above are derived):

Group	Factors (0-9)	Group avg
Threat agent	skill 6 · motive 8 · opportunity 7 · size 6	6.75
Vulnerability	ease-of-discovery 8 · ease-of-exploit 9 · awareness 7 · detectability 6	7.50
Technical impact	confidentiality 9 · integrity 8 · availability 2 · accountability 8	6.75
Business impact	financial 8 · reputation 8 · non-compliance 8 · privacy 8	8.00

Likelihood = avg(threat-agent + vulnerability) = **7.1 (High)**; Impact = avg(technical + business) = **7.4 (High)**.

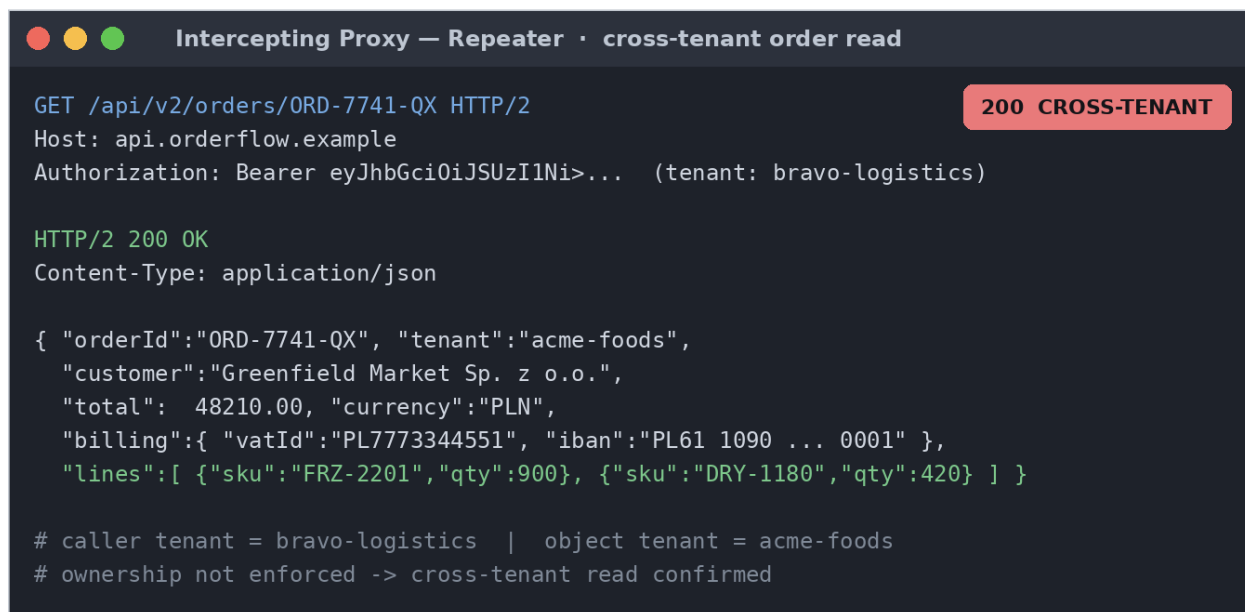
Description. The order resource GET|PUT /api/v2/orders/{orderId} authenticates the caller but does not verify that the order belongs to the caller's tenant. An authenticated user in tenant *bravo-logistics* can supply an order identifier owned by tenant *acme-foods* and receive — or overwrite — the full order, including customer identity, VAT ID, IBAN, and line items. Order identifiers are short and sequential enough to enumerate, so the issue scales from targeted access to bulk cross-tenant harvesting. The list endpoint (GET /api/v2/orders) is correctly tenant-scoped, which confirms this is a per-object authorisation gap on the detail/update path rather than a platform default.

Business Impact. Direct breach of the multi-tenant boundary — the product's core security promise. Cross-tenant disclosure of regulated commercial and personal data (customer identity, tax and bank details), and cross-tenant integrity impact (a competitor tenant can alter another's orders). Reportable under GDPR; material contractual and reputational exposure.

Proof of Concept. As challenger carol@bravo-logistics, requesting an order owned by acme-foods:

```
GET /api/v2/orders/ORD-7741-QX HTTP/2
Host: api.orderflow.example
Authorization: Bearer <carol@bravo-logistics token>

HTTP/2 200 OK
{ "orderId": "ORD-7741-QX", "tenant": "acme-foods", "customer": "Greenfield Market ...",
  "billing": { "vatId": "PL7773344551", "iban": "PL61 1090 ... 0001" }, ... }
```



```
GET /api/v2/orders/ORD-7741-QX HTTP/2
Host: api.orderflow.example
Authorization: Bearer eyJhbGciOiJSUzI1Ni>... (tenant: bravo-logistics)

HTTP/2 200 OK
Content-Type: application/json

{ "orderId":"ORD-7741-QX", "tenant":"acme-foods",
  "customer":"Greenfield Market Sp. z o.o.",
  "total": 48210.00, "currency":"PLN",
  "billing":{"vatId":"PL7773344551", "iban":"PL61 1090 ... 0001" },
  "lines":[ {"sku":"FRZ-2201","qty":900}, {"sku":"DRY-1180","qty":420} ] }

# caller tenant = bravo-logistics | object tenant = acme-foods
# ownership not enforced -> cross-tenant read confirmed
```

Figure — Cross-tenant read confirmed — caller tenant *bravo-logistics* receives an order owned by *acme-foods* with HTTP 200.

A PUT to the same path with a modified body was likewise accepted (HTTP 200), confirming cross-tenant write. Enumeration of the ORD-####-XX space returned other tenants' orders at the same rate.

Recommendation. Enforce server-side ownership validation on every `/orders/{id}` operation: derive the tenant from trusted session state and reject (404/403, fail-closed) any object whose tenant does not match. Apply the same check to all object-addressed endpoints (`/invoices`, `/shipments`, `/customers`). Add automated cross-tenant authorisation tests to the CI pipeline to prevent regression, and treat non-sequential, unguessable identifiers as defence-in-depth (not a substitute for the check).

References. OWASP ASVS v4.0.3 V4.1.1/V4.2.1/V4.2.2; OWASP API Security Top 10 2023 API1; OWASP Top 10 2021 A01; WSTG-ATHZ-04; CWE-639, CWE-863.

5.2. F-002 — Broken function-level authorisation: low-privilege user reaches admin export

Field	Value
Risk Rating	High
Calculation	High (Likelihood 6.5) × Medium (Impact 5.5) = High
Affected system	api.orderflow.example — /api/v2/admin/export
CWE	CWE-285, CWE-862

OWASP factor scores (Likelihood 6.5 / Impact 5.5):

Group	Factors (0-9)	Group avg
Threat agent	skill 5 · motive 8 · opportunity 7 · size 5	6.25
Vulnerability	ease-of-discovery 7 · ease-of-exploit 8 · awareness 6 · detectability 6	6.75
Technical impact	confidentiality 9 · integrity 3 · availability 2 · accountability 7	5.25
Business impact	financial 6 · reputation 7 · non-compliance 6 · privacy 4	5.75

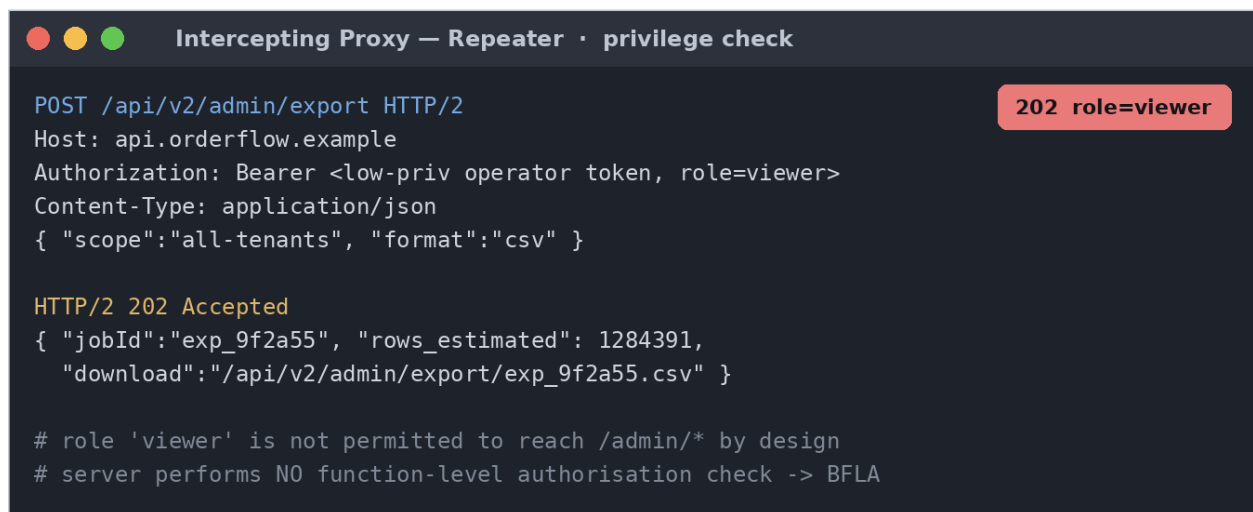
Description. The bulk-export function is hidden from the UI for non-admin users, but the API endpoint enforces no server-side role check. A viewer-role token can invoke POST /api/v2/admin/export with scope: all-tenants and receive a job that produces a CSV of every tenant's orders. The admin console correctly gates the button; the API does not gate the call.

Business Impact. A single low-privilege account can exfiltrate the entire multi-tenant dataset in one authorised request — compounding F-001 from targeted to wholesale. Severe confidentiality and regulatory impact.

Proof of Concept. As low-privilege dave@acme-foods (role viewer):

```
POST /api/v2/admin/export HTTP/2
Authorization: Bearer <viewer token>
{ "scope": "all-tenants", "format": "csv" }

HTTP/2 202 Accepted
{ "jobId": "exp_9f2a55", "rows_estimated": 1284391, "download": "/api/v2/admin/export/exp_9f2a55.csv" }
```



```
Intercepting Proxy — Repeater · privilege check

POST /api/v2/admin/export HTTP/2
Host: api.orderflow.example
Authorization: Bearer <low-priv operator token, role=viewer>
Content-Type: application/json
{ "scope": "all-tenants", "format": "csv" }

HTTP/2 202 Accepted
{ "jobId": "exp_9f2a55", "rows_estimated": 1284391,
  "download": "/api/v2/admin/export/exp_9f2a55.csv" }

# role 'viewer' is not permitted to reach /admin/* by design
# server performs NO function-level authorisation check -> BFLA
```

Figure — A viewer-role token is accepted at an admin-only endpoint; the server performs no function-level authorisation check.

Recommendation. Enforce role-based authorisation server-side on every `/admin/*` route with deny-by-default; derive roles from trusted session claims, not client input. Reject scope: `all-tenants` unless the caller holds an explicit platform-admin entitlement. Add negative authorisation tests for each privileged route.

References. OWASP ASVS v4.0.3 V4.1.3/V4.1.5; OWASP API Security Top 10 2023 API5; OWASP Top 10 2021 A01; WSTG-ATHZ-02; CWE-285, CWE-862.

5.3. F-003 — Unauthenticated actuator endpoint discloses application secrets

Field	Value
Risk Rating	High
Calculation	High (Likelihood 8.0) × Medium (Impact 5.8) = High
Affected system	api.orderflow.example — /actuator/*
CWE	CWE-200, CWE-16, CWE-215

OWASP factor scores (Likelihood 8.0 / Impact 5.8):

Group	Factors (0-9)	Group avg
Threat agent	skill 3 · motive 8 · opportunity 9 · size 8	7.00
Vulnerability	ease-of-discovery 9 · ease-of-exploit 9 · awareness 8 · detectability 5	7.75
Technical impact	confidentiality 9 · integrity 4 · availability 2 · accountability 6	5.25
Business impact	financial 6 · reputation 7 · non-compliance 7 · privacy 5	6.25

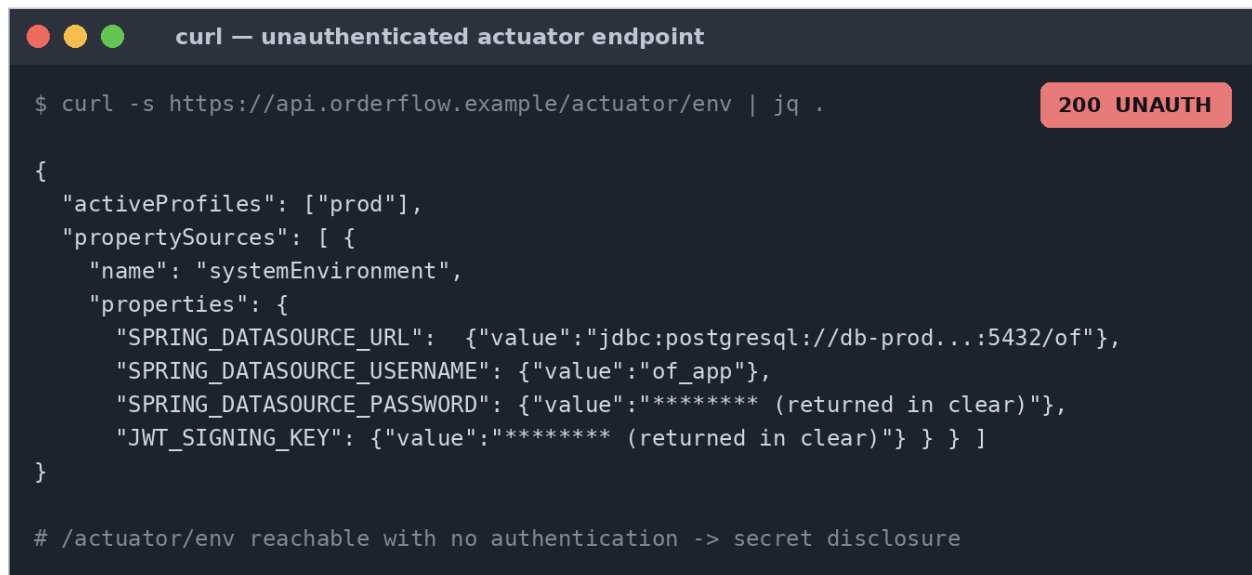
Read alone this is "an exposed endpoint", but it leaks the JWT signing key — see the primary attack chain in Section 2, which is why it is treated as incident-grade.

Description. A Spring Boot Actuator surface is exposed on the API origin with no authentication. /actuator/env returns the full environment, including the database connection string, database credentials, and the JWT signing key in clear. /actuator/health and /actuator/mappings further disclose internal topology and every route.

Business Impact. Disclosure of the JWT signing key alone is critical-adjacent: it would let an attacker forge valid session tokens for any tenant and role, converting this into full platform compromise (and amplifying F-001/F-002). Database credentials disclosure risks direct data-layer access if the database is reachable from any foothold. This is reachable by anyone on the internet with a single unauthenticated request.

Proof of Concept.

```
$ curl -s https://api.orderflow.example/actuator/env | jq .
... "SPRING_DATASOURCE_PASSWORD": {"value":"***** (returned in clear)"},
    "JWT_SIGNING_KEY": {"value":"***** (returned in clear)"} ...
```



```
curl — unauthenticated actuator endpoint

$ curl -s https://api.orderflow.example/actuator/env | jq .

{
  "activeProfiles": ["prod"],
  "propertySources": [ {
    "name": "systemEnvironment",
    "properties": {
      "SPRING_DATASOURCE_URL": { "value": "jdbc:postgresql://db-prod...:5432/of"},
      "SPRING_DATASOURCE_USERNAME": { "value": "of_app"},
      "SPRING_DATASOURCE_PASSWORD": { "value": "***** (returned in clear)"},
      "JWT_SIGNING_KEY": { "value": "***** (returned in clear)" } } } ]
}

# /actuator/env reachable with no authentication -> secret disclosure
```

200 UNAUTH

Figure — The unauthenticated `/actuator/env` response returns datasource credentials and the JWT signing key in clear.

Recommendation. Treat this as incident-grade. Immediately: restrict actuator to an internal management port not exposed at the edge, require authentication, and expose only health with minimal detail. **Rotate the JWT signing key and the database credentials** on the assumption they are compromised, and review access logs for prior reads of `/actuator/env` (Section 7). Add an edge deny-rule for `/actuator/*` as defence-in-depth.

References. OWASP ASVS v4.0.3 V14.3.2/V14.1.x; OWASP Top 10 2021 A05; WSTG-CONF-05; CWE-200, CWE-16, CWE-215.

5.4. F-004 — SSRF via webhook URL-preview (metadata backstopped)

Field	Value
Risk Rating	Medium
Calculation	Medium (Likelihood 5.2) × High (Impact 6.0) = Medium
Affected system	api.orderflow.example — /api/v2/webhooks/preview
CWE	CWE-918

Description. The webhook "preview" feature fetches a user-supplied URL server-side to render a preview. The fetcher follows arbitrary schemes and hosts. Requests to the cloud metadata IP (169.254.169.254) are blocked by an egress policy (a good backstop), but arbitrary external hosts *are* reached and the response body is read back — confirming full, non-blind SSRF against any destination the egress policy does not explicitly block. Internal-range destinations reachable from the API's network position remain a risk if the egress allow-list is incomplete.

Business Impact. An attacker can use the platform as a request proxy from its trusted network position (port-scan internal services, reach unprotected internal endpoints, exfiltrate via DNS), and read the responses. The metadata backstop prevents the worst case (cloud credential theft) *today* but the design relies on that single control.

Proof of Concept.

```
POST /api/v2/webhooks/preview { "url":"http://169.254.169.254/latest/meta-data/..." }
-> 502 upstream fetch blocked by egress policy (metadata blocked – good)

POST /api/v2/webhooks/preview { "url":"http://poc-8831.oob.example/" }
-> out-of-band DNS + HTTP callback received from cloud egress IP (SSRF confirmed)
```

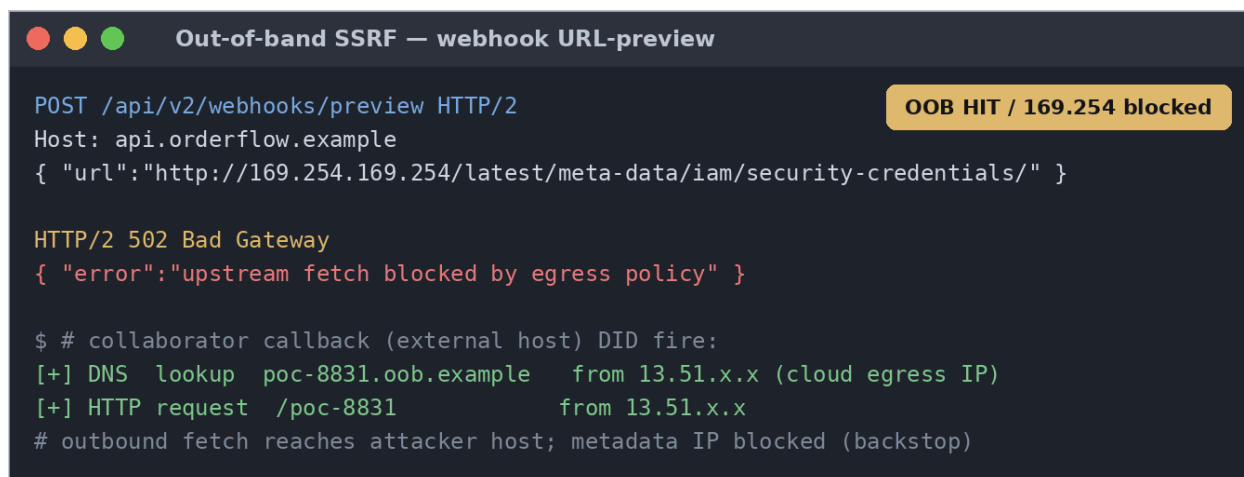


Figure — Out-of-band callback confirms server-side fetch of an attacker URL; the metadata IP is blocked by egress policy as a backstop.

Recommendation. Replace host-based blocking with a strict allow-list of permitted destinations, resolve and re-validate the target after DNS resolution (prevent rebinding), disable redirects, and never return the fetched body to the client. Keep the metadata-IP egress block, and prefer IMDSv2 with hop-limit 1.

References. OWASP ASVS v4.0.3 V12.6.1/V5.2.6; OWASP API Security Top 10 2023 API7; OWASP Top 10 2021 A10; WSTG-INPV-19; CWE-918.

5.5. F-005 — Stored XSS in order note rendered in the operator console

Field	Value
Risk Rating	Medium
Calculation	Medium (Likelihood 5.0) × Medium (Impact 4.5) = Medium
Affected system	console.orderflow.example
CWE	CWE-79

Description. The `internalNote` field on an order is stored without sanitisation and rendered unescaped in the operator console. A customer-side user can plant a payload that executes in the operator's authenticated origin when staff open the order. The session cookie is `HttpOnly` (so the token is not directly stealable), but script running as the operator can still perform authenticated actions on their behalf.

Business Impact. Cross-privilege escalation vector: a low-trust customer input executes code in a high-trust operator session. Combined with F-002, an attacker could script a privileged export from the victim operator's browser.

Proof of Concept.

```
PUT /api/v2/orders/ORD-5520-KP { "internalNote": "<img src=x onerror=alert(document.domain)>"
  }
# operator opens the order in console.orderflow.example -> script executes in admin origin
```

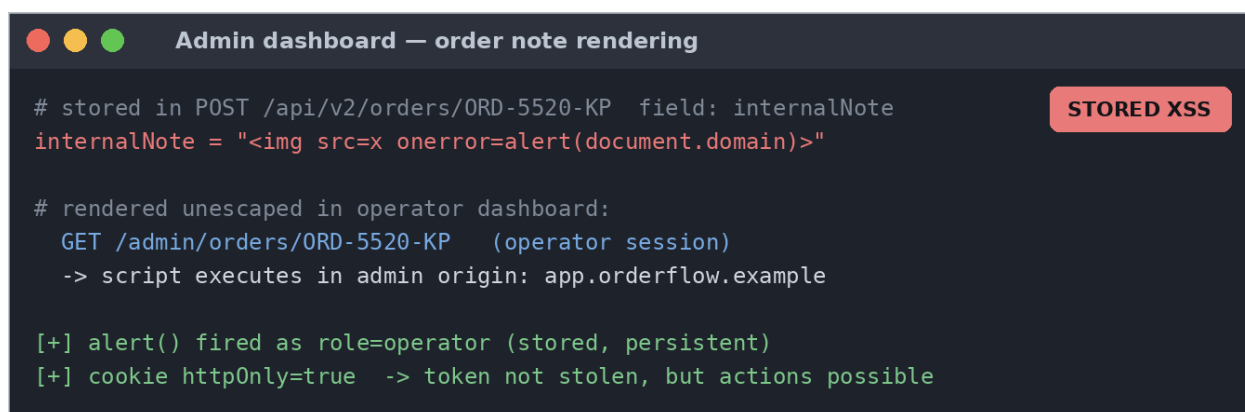


Figure — Stored payload in an order note executes in the operator console origin when staff view the order.

Recommendation. Contextually output-encode all user-controlled data on render, sanitise rich text server-side against an allow-list, and deploy a strict Content-Security-Policy (see F-007) as defence-in-depth to neutralise injected script.

References. OWASP ASVS v4.0.3 V5.3.3/V5.3.4; OWASP Top 10 2021 A03; WSTG-CLNT-01; CWE-79.

5.6. F-006 — Long-lived refresh token stored in browser localStorage

Field	Value
Risk Rating	Medium
Calculation	Medium (Likelihood 4.9) × Medium (Impact 4.3) = Medium
Affected system	app.* / console.*
CWE	CWE-522, CWE-539

Description. Both front ends persist the session — including a 30-day **refresh token** — in localStorage, which is readable by any JavaScript on the origin. Access tokens are short-lived, but the refresh token is a long-lived credential; any XSS (e.g. F-005) or malicious dependency on the origin yields a 30-day session takeover rather than a transient one.

Business Impact. Turns any transient client-side compromise into durable account access. Materially raises the impact of F-005 and of any future third-party script incident.

Proof of Concept.

```
> JSON.parse(localStorage.getItem('of.session'))
{ accessToken:"eyJ...", refreshToken:"eyJ...(valid 30 days)...", roles:["viewer"], tenant:"bravo-logistics" }
```

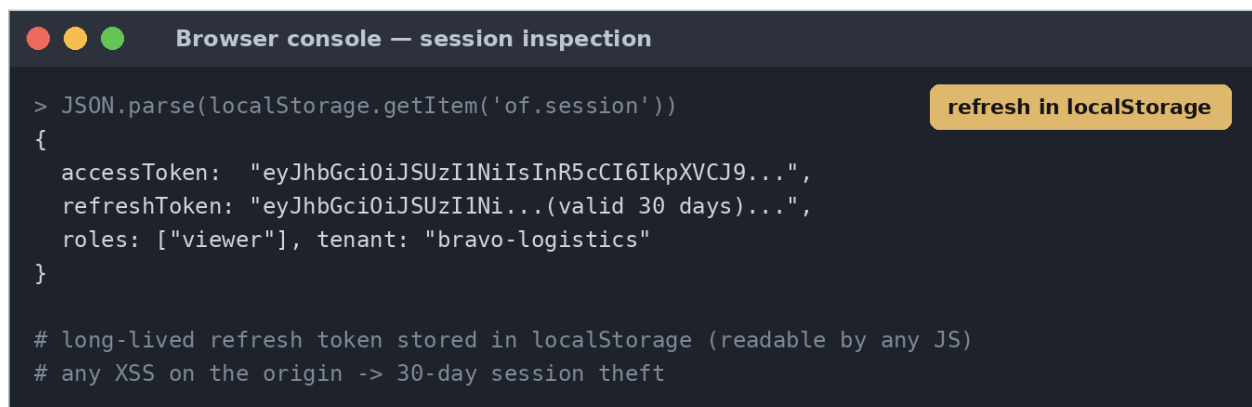


Figure — The 30-day refresh token is retrievable from localStorage via the browser console — readable by any script on the origin.

Recommendation. Store refresh tokens in a Secure; HttpOnly; SameSite cookie, or adopt a backend-for-frontend pattern so long-lived credentials never reach the browser. Shorten refresh-token lifetime, bind it to a client fingerprint, and support server-side revocation.

References. OWASP ASVS v4.0.3 V3.5.2/V3.5.3/V8.2.x; OWASP Top 10 2021 A07; RFC 9700; CWE-522, CWE-539.

5.7. F-007 — Missing HTTP security headers across all origins

Field	Value
Risk Rating	Low
Calculation	High (Likelihood 7.2) × Low (Impact 2.6) = Low
Affected system	all web origins
CWE	CWE-1021, CWE-693

Description. Application responses omit Content-Security-Policy, X-Frame-Options/frame-ancestors, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, and Strict-Transport-Security. The Keycloak login origin is correctly hardened; the application origins are not. On its own this creates no vulnerability, but it removes the browser-side controls that would contain other issues — most directly, the absence of CSP amplifies F-005.

Business Impact. Clickjacking of authenticated console pages; no CSP backstop against XSS (compounds F-005); referrer leakage of identifiers (e.g. orderId from F-001); MIME-sniffing risk; a TLS-stripping window without HSTS.

Proof of Concept. `curl -sI` against each origin returns only content-type and cache-control; a CORS probe with an arbitrary Origin returns no Access-Control-Allow-Origin (CORS is *not* misconfigured — a genuine negative worth recording).

Recommendation. Set a baseline at the shared edge and in the framework: a strict CSP, frame-ancestors 'none', nosniff, Referrer-Policy: strict-origin-when-cross-origin, a least-privilege Permissions-Policy, and HSTS with a defined max-age. Validate centrally.

References. OWASP ASVS v4.0.3 V14.4.x; OWASP Top 10 2021 A05; OWASP Secure Headers Project; CWE-1021, CWE-693.

5.8. F-008 — Verbose error handling discloses stack traces

Field	Value
Risk Rating	Low
Calculation	Medium × Low = Low
Affected system	api.orderflow.example
CWE	CWE-209

Description. Malformed input to several API endpoints returns an unhandled exception with a full stack trace, disclosing framework versions, internal package structure, and SQL fragments. This is reconnaissance-grade information that shortens an attacker's path to framework- and query-specific attacks.

Business Impact. No direct compromise; accelerates targeting and reveals internal structure. Best read with F-003 (which discloses far more).

Proof of Concept. A non-numeric value in a numeric query parameter (?limit=abc) returns HTTP 500 with a Java stack trace and the offending SQL statement in the body.

Recommendation. Return generic error bodies to clients; log detail server-side only. Add a global exception handler that maps unexpected errors to a correlation ID without leaking internals.

References. OWASP ASVS v4.0.3 V7.4.1; OWASP Top 10 2021 A05; WSTG-ERRH-01; CWE-209.

5.9. F-009 — Dependency & transport posture (current and patched)

Field	Value
Risk Rating	Informational (positive posture note)
Affected system	estate

Description. The client-side framework versions observed are current and patched; no known-vulnerable framework build was found, and two relevant published bypasses were tested and do not work. TLS is 1.2/1.3 only with no legacy protocols or weak ciphers, and source maps are not exposed. This is a genuine strength and is recorded so the client can see what *is* working.

Recommendation. Maintain patch cadence and dependency monitoring; validate server-side package versions via SBOM where black-box confirmation is not possible.

5.10. F-010 — External attack-surface enumeration: unscoped assets

Field	Value
Risk Rating	Informational
Affected system	estate (public sources)
CWE	CWE-200, CWE-668

Description. Passive reconnaissance (Section 3) surfaced assets on the same public estate that were **not** in the agreed scope: a staging.* origin and a reports.* host running an exposed BI tool, plus a public S3 bucket that exists (403 on listing). These were not tested, but they share the platform's DNS and likely its identity realm — so any systemic flaw found in scope may also apply to them. The BI host is the most material: reporting surfaces are a common location for aggregated, cross-tenant data.

Recommendation. Maintain a current external asset inventory; bring the BI/reporting host and staging origin into a formal test or monitoring programme; confirm the S3 bucket's policy denies public listing and object read. Avoid predictable, business-descriptive naming for internet-facing cloud resources.

References. OWASP ASVS v4.0.3 V1.1.2; OWASP WSTG-INFO-01/02/04; MITRE ATT&CK TA0043; CWE-200, CWE-668.

6. MITRE ATT&CK Mapping

Mapping the exploited behaviours to ATT&CK gives the client's blue team a shared language for detection and lets them relate these findings to their existing coverage.

Tactic	Technique	Finding
Reconnaissance	T1590 Gather Victim Network Info; T1596 Search Open Technical Databases	F-010, §2
Initial Access	T1190 Exploit Public-Facing Application	F-001, F-003
Credential Access	T1552.001 Credentials in Files (env endpoint)	F-003
Discovery	T1526 Cloud Service Discovery; T1087 Account Discovery	F-002, §2
Collection	T1213 Data from Information Repositories	F-001, F-002
Exfiltration	T1567 Exfiltration Over Web Service	F-002 (bulk export)
Lateral / Pivot	T1090 Proxy (SSRF as request proxy)	F-004

7. Indicators of Compromise & Detection Opportunities

Because F-001 and F-003 are trivially exploitable and may have been reached before this test, this section gives the client concrete signatures to hunt for prior abuse and to detect future attempts. Each is written to drop into a SIEM or WAF.

F-001 / F-002 — cross-tenant & privilege abuse.

- *Hunt*: API access log where the authenticated token's tenant claim $\neq$ the responded object's tenant, on 2xx responses to `/api/v2/orders/*`, `/invoices/*`, `/admin/*`.
- *Signature (pseudo-SIEM)*: `path:/api/v2/admin/* AND token.role:"viewer" AND status:2xx` → alert (should never match).
- *Signature*: a single token issuing >N distinct `orderId` reads per minute → enumeration alert.

F-003 — actuator secret disclosure.

- *Hunt (retrospective)*: any request to `/actuator/env`, `/actuator/mappings`, `/actuator/configprops` in historical access logs — treat any hit as potential key compromise.
- *Signature*: `path:/actuator/* AND NOT src_ip:<mgmt range>` → high-severity alert; block at edge.
- *IoC*: forged JWTs would validate but may show anomalous `iat/iss` or `roles/tenants` that never authenticated via Keycloak — cross-check API tokens against IdP issuance logs.

F-004 — SSRF.

- *Signature*: outbound egress from API subnets to non-allow-listed hosts, or DNS lookups for domains not in the app's known dependency set, correlated with `/webhooks/preview` requests.

F-005 — stored XSS.

- *Hunt*: stored `internalNote`/free-text fields containing `<`, `onerror=`, `javascript:`, `<script;` review before rendering in the console.

F-006 — token theft.

- *Signature*: the same `refreshToken` presented from two distinct IPs/user-agents within its lifetime → session-anomaly alert.

8. Remediation Roadmap

Prioritised so the client gets maximum risk reduction per unit of effort.

Immediate (0-72h) — incident-grade.

1. F-003 — Restrict `/actuator/*`, rotate the JWT signing key and DB credentials, review logs for prior `/actuator/env` reads.
2. F-001 — Add the per-object tenant ownership check on `/orders/{id}` (and siblings); deploy behind a feature flag if needed for speed.

Short term (1-2 weeks).

3. F-002 — Server-side role checks on all `/admin/*` routes, deny-by-default.
4. F-004 — Destination allow-list + no response read-back on the webhook fetcher.
5. F-005 — Output-encode/sanitise order notes.

Medium term (1-2 months).

6. F-006 — Move refresh tokens to `HttpOnly` cookies / BFF.
7. F-007 — Roll out the security-header baseline at the edge.
8. F-008 — Global exception handler with generic client errors.

Programmatic (ongoing).

9. Add cross-tenant and negative-authorisation tests to CI (prevents F-001/F-002 regression — the highest-leverage single control).
10. Bring the unscoped `reports.*` / `staging.*` hosts (F-010) into scope; re-test after remediation.

9. Appendix A — Tools Used

Phase	Tooling
Passive recon	subfinder, assetfinder, crt.sh (CT logs), amass -passive, WHOIS/RDAP, Wayback
Cloud attribution	Service-tag & global-namespace DNS-existence checks (passive), S3 bucket existence probes
Liveness / fingerprint	httpx, katana, whatweb, JS bundle inspection, OIDC discovery
Transport / headers	testssl.sh, sslyze, nmap (ssl/http- NSE), curl
Web app testing	Intercepting proxy (Repeater, Intruder, Collaborator) for manual verification, OOB SSRF, request replay
Injection	sqlmap (negative on parameterised paths)
Auth / crypto	JWT decode/forged tests, offline session-secret dictionary attempt, OIDC flow analysis

10. Appendix B — Glossary

Term	Definition
BOLA	Broken Object Level Authorization — the server fails to verify that the caller owns the requested object (cf. IDOR). The root cause of F-001.
BFLA	Broken Function Level Authorization — a privileged function lacks a server-side access-control check, so a low-privilege caller can invoke it. The root cause of F-002.
SSRF	Server-Side Request Forgery — the server is coerced into making attacker-controlled outbound requests from its own trusted network position.
IDOR	Insecure Direct Object Reference — a request references an object by identifier without an ownership check; the practical symptom of a BOLA flaw.
JWT	JSON Web Token — a signed, self-contained token carrying identity and authorisation claims. If the signing key leaks (F-003), tokens can be forged.
BFF	Backend-for-Frontend — a server-side component that holds tokens on the browser's behalf, so long-lived credentials never reach client-side storage.
Actuator	Spring Boot's operational endpoint surface (/actuator/*); useful internally, dangerous when exposed unauthenticated (F-003).
CSP	Content-Security-Policy — an HTTP header that restricts which sources a page may load, containing XSS and injection (F-005, F-007).
HSTS	HTTP Strict-Transport-Security — a header that forces browsers to use HTTPS, closing the TLS-stripping window (F-007).
OWASP Risk Rating	The Likelihood × Impact framework used as the primary severity model throughout this report.
CVE / CWE	Public catalogues of, respectively, specific known vulnerabilities and general weakness types.
MITRE ATT&CK	A knowledge base of adversary tactics and techniques, used in Section 6 to map exploited behaviours to detection language.